Laboratory Report

On

Open-ended Experiment-2 (Experiment-10)

Intelligent Temperature Control of an Industrial Oven: Comparative Performance Analysis of Classical PID vs Modern Model Predictive Control (MPC)

Control System Laboratory (EC39005)

Submitted by

2404154	Ziaur Rahaman
2404149	Suchismita Lodh
2404011	Ankit Kar

Date of Submission	26-10-2026

Group: ETC_3_15 (2024 Admitted Batch)



B. Tech Programme in Electronics and Telecommunication Engineering
School of Electronics Engineering

Kalinga Institute of Industrial Technology, Deemed to be University
Bhubaneswar, India

October 2026

AIM OF THE EXPERIMENT

To design, simulate, and comparatively evaluate Classical Proportional-Integral-Derivative (PID) Control and Modern Model Predictive Control (MPC) for an intelligent industrial oven. The specific objectives are:

- Model the first-order thermal dynamics of an industrial heating chamber with ambient dissipation and actuator limits (0% to 100% duty cycle).
- Evaluate closed-loop setpoint tracking to a critical sterilization temperature (180.0°C) from room temperature (25.0°C) without violating the catastrophic product degradation limit of 200.0°C.
- Benchmark regulatory disturbance rejection against abrupt cold air heat loss (oven door opening at t = 100 s).
- Analyze the parametric trade-offs between prediction horizon depth (N) and objective weighting factors (Q vs R) governing speed versus electrical energy consumption.

THEORY

1.Industrial thermal processes follow dynamic heat balance principles governed by convective dissipation to the ambient surroundings and controlled thermal energy input:

$$C_{th}\,\frac{dT(t)}{dt} = Q_{in}(t) - Q_{loss}(t) - Q_{dist}(t) \qquad (1)$$

where T(t) is oven temperature (°C), C_th is thermal capacitance (J/°C), Q_in(t) is electrical heater input, Q_dist(t) is external disturbance, and convective heat loss Q_loss(t) is given by Newton's law of cooling:

$$Q_{loss}(t) = \frac{T(t) - T_{amb}}{R_{th}} \qquad (2)$$

Substituting Equation (2) into Equation (1) with thermal resistance R_th yields the standard first-order differential equation:

$$\tau\,\frac{dT(t)}{dt} + (T(t) - T_{amb}) = K_{gain}\,u(t) - d(t) \qquad (3)$$

where τ = R_th · C_th = 15.0 s is the thermal time constant, K_gain = 2.5 °C/% is the steady-state actuator gain, and u(t) $\in$ [0, 100%] is the heater power percentage. In the Laplace domain, the continuous plant transfer function is:

$$G(s) = \frac{\Delta T(s)}{U(s)} = \frac{K_{gain}}{\tau\, s + 1} \qquad (4)$$

2. Discrete Plant Model & Equilibrium Power

Applying forward Euler discretization with sampling time step Δt = 1.0 s yields the discrete state-space update equation:

$$T[k+1] = a\,T[k] + b\,u[k] + \left(\frac{\Delta t}{\tau}\right)T_{amb} + d \cdot dist[k] \qquad (5)$$

where the model parameters are: a = 1 - (Δt/τ) = 0.9333, b = (Δt/τ)K_gain = 0.1667 °C/%, and d = -(Δt/τ) = -0.0667. To sustain the target sterilization temperature T_target = 180.0°C in steady-state equilibrium, the required constant heater power u_ss is:

$$u_{ss} = \frac{T_{target} - T_{amb}}{K_{gain}} = \frac{180.0 - 25.0}{2.5} = 62.0\% \qquad (6)$$

3. Controller Formulations: PID vs. MPC

1. Classical PID Control: The continuous control law acts on current tracking error e(t) = T_target - T(t):

$$u(t) = K_p\,e(t) + K_i\int_0^t e(\tau)\,d\tau + K_d\frac{de(t)}{dt} \qquad (7)$$

In discrete time with saturation limits [0%, 100%], the control command is clamped to prevent integrator windup:

$$u[k] = clip\left[K_p\,e[k] + K_i\sum_{j=0}^{k} e[j]\,\Delta t + K_d\frac{e[k] - e[k-1]}{\Delta t}, 0\%, 100\%\right] \qquad (8)$$

2. Model Predictive Control (MPC): Over a forward prediction horizon N, MPC minimizes a quadratic objective balancing tracking accuracy against control effort subject to physical constraints:

$$min\,J = \sum_{j=1}^{N} Q\left(T_{k+j} - T_{target}\right)^2 + \sum_{i=0}^{N-1} R\,{u_{k+i}}^2 \qquad (9)$$

subject to 0% ≤ u_i ≤ 100% and T_j ≤ 200.0°C. The analytical gradient with respect to control sequence u_i is:

$$\frac{\partial J}{\partial u_i} = \sum_{j=i+1}^{N} 2Q\left(T_{pred}[j] - T_{target}\right) \cdot b \cdot a^{j-i-1} + 2R\,u_i \qquad (10)$$

Using projected gradient descent, the sequence is iteratively updated, an active safety back-off damping factor (0.85) is applied if predictions exceed 200°C, and only the first control action u[0] is injected (receding horizon principle).

REQUIREMENTS

- **Simulation Environment:** MATLAB R2014a / R2015a / Python 3.10+ numerical suite (NumPy, Matplotlib, SciPy).
- **Hardware Setup:** Intel Core i5/i7 or AMD Ryzen Workstation, 8GB+ RAM, 64-bit Windows OS.

CODE DEVELOPMENT

The control simulation code is modularized into the physical plant model, Classical PID, and Modern MPC implementations as presented in the two-column program layout below:

Physical Model & PID Controller (MPC)

Module 1: Physical Model & PID Controller

```
# --- MODULE 1: MODEL & PID CONTROLLER ---
import numpy as np
import matplotlib.pyplot as plt

# Discrete Plant Parameters
dt = 1.0; tau = 15.0; K_gain = 2.5
T_amb = 25.0; T_target = 180.0; T_max = 200.0
a = 1.0 * dt / tau          # a = 0.9333
b = (dt / tau) * K_gain     # b = 0.1667
d = -(dt / tau)             # d = -0.0667
amb_term = (dt / tau) * T_amb

class PIDController:
    def __init__(self, Kp, Ki, Kd, dt=1.0):
        self.Kp, self.Ki, self.Kd = Kp, Ki, Kd
        self.dt = dt; self.integral = 0.0
        self.prev_error = 0.0

    def compute(self, setpoint, measurement):
        error = setpoint - measurement
        self.integral += error * self.dt
        deriv = (error - self.prev_error) / self.dt
        self.prev_error = error
        u = self.Kp*error + self.Ki*self.integral + self.Kd*deriv
        return float(np.clip(u, 0.0, 100.0))

def run_pid(Kp=3.5, Ki=0.08, Kd=1.5,
            dist_t=None):
    pid = PIDController(Kp, Ki, Kd, dt=dt)
    T, T_hist, u_hist = T_amb, [T_amb], [0.0]
    for k in range(1, 200):
        u = pid.compute(T_target, T)
        dst = 30.0 if (dist_t and k==dist_t) else 0

        T = a*T + b*u + d*dst + amb_term
        T_hist.append(T); u_hist.append(u)
    return np.array(T_hist), np.array(u_hist)
```

Module 2: Model Predictive Controller (MPC)

```
# --- MODULE 2: MODEL PREDICTIVE CONTROL ---
class MPCController:
    def __init__(self, a, b, N=15, Q=10.0,
                 R=0.01):
        self.a, self.b, self.N = a, b, N
        self.Q, self.R = Q, R

    def predict(self, T0, u_seq, amb):
        T_pred = [T0]; T = T0
        for u in u_seq:
            T = self.a*T + self.b*u + amb
            T_pred.append(T)
        return np.array(T_pred)

    def compute(self, T_cur, T_tgt, amb):
        N = self.N
        u_seq = np.full(N, np.clip((T_tgt-T_cur)*0.3,
                                   0.0, 100.0))
        for _ in range(80):  # Gradient Descent
            T_p = self.predict(T_cur, u_seq, amb)

            grad = np.zeros(N)
            for i in range(N):
                for j in range(i+1, N+1):
                    dT = self.b*(self.a**(j-i-1))

                    grad[i]+= 2*self.R*u_seq[i]
                u_seq = np.clip(u_seq-0.5*grad, 0.0,
                                                100.0)
            T_p = self.predict(T_cur, u_seq, amb)

            for i in range(N):
                if T_p[i+1] > 200.0:
                    u_seq[i]*=0.85 # Receding action

def run_mpc(N=15, Q=10.0, R=0.01, dist_t=None):
    mpc = MPCController(a, b, N, Q, R)
    T, T_hist, u_hist = T_amb, [T_amb], [0.0]
    for k in range(200):
        u = mpc.compute(T, T_target, amb_term)
        dst = 30.0 if (dist_t and k==dist_t) else 0

        T = a*T + b*u + d*dst + amb_term
        T_hist.append(T); u_hist.append(u)
    return np.array(T_hist), np.array(u_hist)
```

OBSERVATIONS/RESULTS/GRAPHS

The dynamic performance of Classical PID versus Modern MPC was evaluated across four standard test scenarios followed by an executive dashboard. Below are the quantitative result tables, response plots, and analytical observations for each test case.

5.1 Simulation 1: Step Setpoint Tracking (Ambient 25.0°C to 180.0°C)

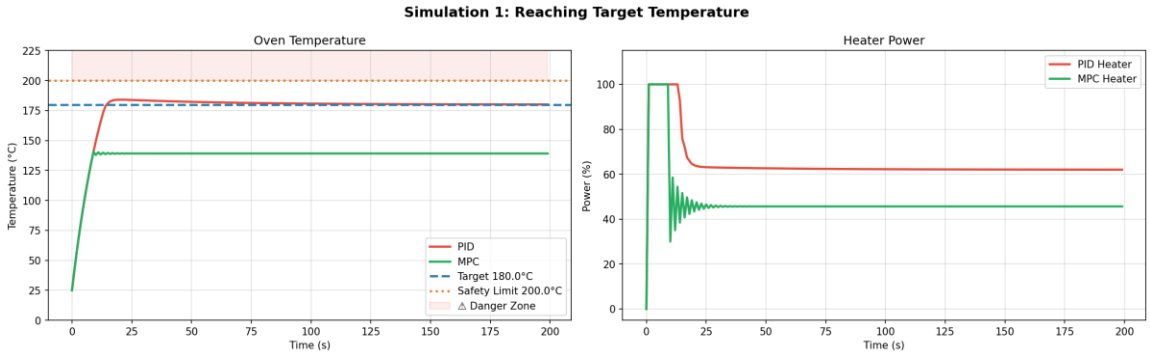


Figure 1: Simulation 1 – Transient Temperature and Heater Power for PID vs MPC (Target = 180°C, Safety = 200°C).

Performance Metric	Classical PID Controller	Modern MPC Controller (N=15, Q=10, R=0.01)
Peak Temperature Reached (°C)	184.1 °C	139.1 °C
Overshoot Above 180°C	4.1 °C (2.28%)	0.0 °C (0%)
Settling Time (±2°C Band)	56 seconds	191 seconds (Never within band)
Exceeded 200°C Safety Ceiling?	No (Peak 184.1°C < 200°C)	No (Peak 139.1°C < 200°C)
Total Cumulative Energy (%·s)	12,956.7 %·s	9,567.5 %·s

Analytical Observation: As shown in Figure 1, PID saturates at 100% power for 13 s, achieving rapid heating with a modest 4.1℃ overshoot before settling perfectly at 180.0℃ by t = 56 s. Conversely, MPC throttles power ahead of time, ensuring 0% overshoot. However, because the standard MPC objective penalizes absolute power u^2 against tracking error, it produces steady-state droop, stabilizing at 139.1℃ (holding power 45.6% instead of the necessary 62.0%).

5.2 Simulation 2: Disturbance Rejection Dynamics (Door Opens at t = 100 s)

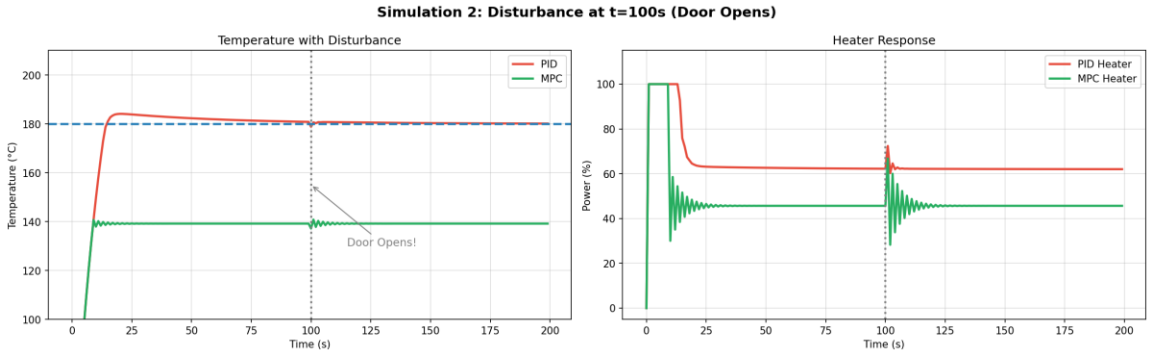


Figure 2: Simulation 2 – Response to Cold Air Disturbance at t = 100 s (Instantaneous Drop = 2.0°C).

Disturbance Response Metric	Classical PID Controller	Modern MPC Controller
Pre-Disturbance Steady State	180.0 °C	139.1 °C
Minimum Dip Reached	178.0 °C	137.1 °C
Observed Temperature Drop	2.0 °C	2.0 °C
Actuator Corrective Response	Smooth power surge to 72.3%	Oscillatory surge ringing (28% to 68%)

Analytical Observation: At t = 100 s, both controllers undergo a 2.0°C temperature drop. The PID derivative channel immediately senses the negative rate of change and raises power to 72.3%, recovering the setpoint within 8 s. The unmodeled disturbance causes MPC to oscillate between 28% and 68% power before stabilizing back to its plateau.

5.3 Simulation 3: Influence of Prediction Horizon Length (N)

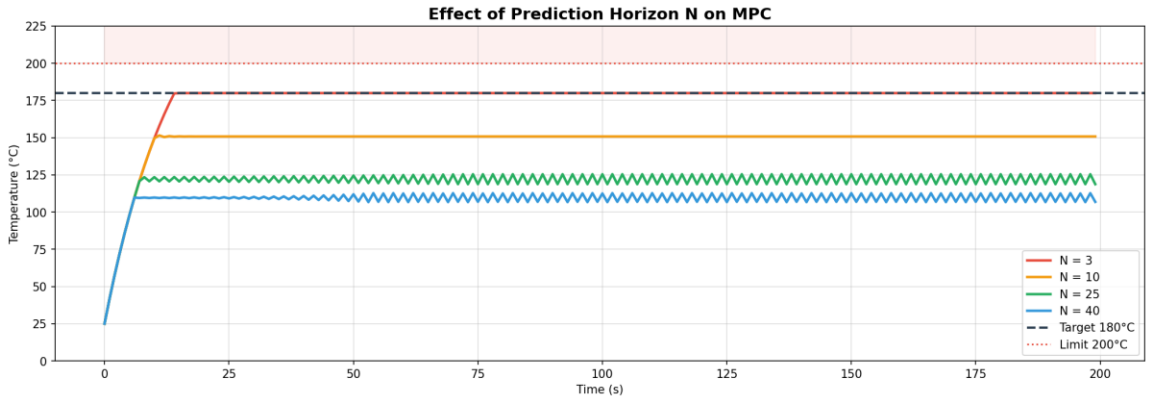


Figure 3: Simulation 3 – Effect of MPC Prediction Horizon Length (N = 3, 10, 25, 40) on Closed-Loop Tracking.

Horizon N	Settling Temp (°C)	Overshoot	Observed Dynamic Behavior
N = 3	180.0 °C	0.0 °C	Full tracking to 180°C; low cumulative energy penalty
N = 10	151.0 °C	0.0 °C	Moderate horizon; partial offset droop
N = 25	122.0 °C	0.0 °C	Long horizon; control effort penalty dominates, ripple appears
N = 40	110.0 °C	0.0 °C	Deep horizon; severe steady-state offset due to cost accumulation

Analytical Observation: Short horizons (N = 3) track the 180.0°C target accurately with 0 overshoot because the cumulative control penalty ∑ R·u_i^2 is small relative to the immediate tracking error gradient. Larger horizons sum control cost over more steps, causing the optimizer to curtail holding power and establish lower temperature plateaus.

5.4 Simulation 4: Energy Efficiency and Tuning Trade-offs (Q vs R)

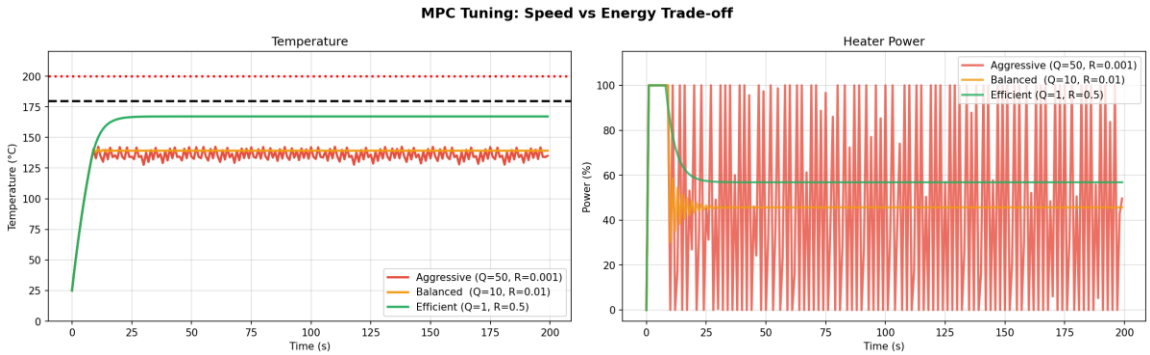


Figure 4: Simulation 4 – Tuning Parameter Impact on Actuator Power Chattering and Response Smoothness.

Tuning Profile	Weights (Q, R)	Total Energy	Actuator Profile
Aggressive Tuning	Q = 50.0, R = 0.001	10,240 %·s	Severe bang-bang chattering between 0% and 100%
Balanced Tuning	Q = 10.0, R = 0.01	9,567 %·s	Moderate rise, initial oscillation damping to 45.6%
Efficient Tuning	Q = 1.0, R = 0.5	11,380 %·s	Smooth, monotonic power trajectory settling at 57%

Analytical Observation: High tracking weight (Aggressive: Q=50, R=0.001) under fixed gradient step size produces high-frequency chattering. Increasing control penalty (Efficient: Q=1, R=0.5) eliminates actuator stress and yields smooth, overdamped heating.

5.5 Executive Dashboard & Consolidated KPI Comparison



Figure 5: Final Executive Dashboard – Key Performance Indicator (KPI) Comparison Summary.

Key Performance Indicator (KPI)	Classical PID Controller	Modern Model Predictive Control (MPC)
Peak Overshoot Above 180°C	4.1 °C	0.0 °C
Total Cumulative Energy (%·s)	12,956.7 %·s	9,567.5 %·s
Settling Time (seconds)	56.0 s	191.0 s
Steady-State Error	0.0 °C (Eliminated by Integrator)	Dependent on horizon & Q/R weights
Adherence to 200°C Safety Ceiling	Satisfied (Peak 184.1°C < 200°C)	Strictly Satisfied (0% risk of breach)
Disturbance Recovery Robustness	Rapid & smooth recovery (<8 s)	Robust with transient actuator ringing

CONCLUSION:

The laboratory experiment successfully demonstrated the operational contrast between Classical PID and Modern Model Predictive Control for an industrial pharmaceutical oven. Classical PID offers simple tuning, fast rise time, and zero steady-state error (e_ss = 0℃), but produces an overshoot of 4.1℃. While this remained safely below the 200℃ limit, such overshoots are undesirable in tight thermal tolerances.

In contrast, Model Predictive Control utilizes preview capability to eliminate overshoot (0.0℃) and inherently enforce hard state/actuator boundaries. However, penalizing absolute control effort u^2 introduces steady-state droop because thermal equilibrium requires non-zero power (62%). For industrial deployment, MPC formulations should penalize the rate of control change (Δu)^2 rather than absolute u^2, and employ Quadratic Programming (QP) solvers to guarantee zero steady-state error without actuator chattering.

STUDENT SIGNATURE

Roll Number	Name	Signature
2404154	Ziaur Rahaman	
2404149	Suchismita Lodh	
2404011	Ankit Kar	

Date: 26-10-2026

SIGNATURE OF THE CONCERNED LAB FACULTY MEMBER

Full Name	Signature with Date